SHARD: Measurement-Driven Performance Engineering of a Spark Pipeline over NYC Taxi Trips

Radesh Govind
p215604@campus.fct.unl.pt
NOVA FCT
Caparica, Portugal

ABSTRACT

This report studies the run time of a Spark pipeline of five operations over 48.7 million New York taxi trips from 2025, on a laptop and on a simulated cluster of workers running on that same laptop. Measuring each part showed two delays. *First*, the same files were read five times, once per operation (about three fifths of the run time). Keeping the cleaned trips in memory had the opposite effect: the pipeline took 32 % to 42 % longer, because building the copy cost more than the reading it saved, and on the cluster the copy did not fit in memory. *Second*, the operation that finds unusual fares regrouped every trip across tasks, sorted it and partly wrote it to disk (about one sixth). Letting each task first sum its own trips made that operation about two fifths faster, and the whole pipeline 16 % faster on the laptop and 9 % on the cluster. A third change, to one join, gave no measurable gain. The run time grows much less than the data, and adding workers on one laptop does not reduce it; both are traced to measured causes.

KEYWORDS

Apache Spark, performance engineering, shuffle, broadcast join, Amdahl's law

1 INTRODUCTION

Many programs must process more data than one processor can handle quickly. Spark is a tool for this: it splits a data job into tasks that run on many processors at the same time. A program that reads data, computes results from it and writes them to files is a *pipeline*. This report studies one pipeline and asks how to make it faster.

The data. The pipeline works on the trips of the New York yellow taxis in 2025 [5]: 48,722,602 trips, one row per trip, in 12 monthly Parquet files (829,973,299 B). Each trip has its start and end time, pickup and drop-off zone, passengers, distance, fare and payment type. Valid zone numbers are 1–263; a lookup table of 265 rows gives each zone its borough. After five validity checks (e.g. a fare above zero), 43,599,832 trips remain.

The five operations. Table 1 lists the five results, each with the question it answers and the reason it takes time. O1 and O3 were chosen from the optional operations because they stress two different costs: O1 regroups every trip for its group statistics, and O3 depends on how Spark chooses to execute a join.

The method. Each improvement follows the loop of Figure 1: measure where the time goes, state a hypothesis about the cause, change one thing, and measure again. A change is kept only if the new measurement is better. Sections 3 and 4 follow these steps; Section 5 tests the final program with more data and more workers.

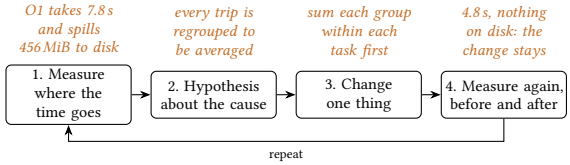


Figure 1: The measure-and-improve loop, with the example of Section 4.2 (laptop).

2 PIPELINE

The pipeline works in four steps: (1) read the 12 monthly files; (2) check the trips against the five rules of M1 (M1 counts the failures, the other operations remove those trips); (3) compute the result of each operation; (4) write the results to files. The operations run one after the other and do not share data, so each one reads the whole year again.

How the program is written. The program is written in Kotlin, using Spark's DataFrame API on the Java virtual machine (JVM). Every step is a built-in Spark expression, with no user-defined functions; DataFrame code and SQL are optimised by the same engine (Catalyst [2]), and the DataFrame form is checked by the compiler. A program option (`-skip-opts`) switches each tested change off, so old and new versions run from one build. No Spark configuration property was changed.

One plan as an example. Figure 2 shows the final plan of the unusual-fare operation (O1) as Spark ran it: (a) at first, (b) after the change of Section 4.2.

3 PROFILING AND BASELINE

The first step of the loop is to measure the pipeline as it is. These first measurements are the *baseline*, and every later change is compared with them.

On which machines? The measurements ran on a laptop and on a simulated cluster (Table 2). A cluster is a group of computers, called workers, that share a job. In the simulated cluster the workers are containers on the same laptop and share its hardware.

How was the time measured?

- *Recorded:* a Kotlin version of the Lab 3 profiling tool [3] records each operation's run time, its stage metrics (data regrouped, spilled) and the final plans. The run time T is the sum of the five operations, without the start of Spark.
- *Alternation:* the laptop is not a steady machine. Whole sessions differed by up to about 40 % on the same code (in one, Low Power

Table 1: The five operations. M = required by the assignment, O = optional, chosen by the author.

Operation	The question it answers	Why it takes time
M1 cleaning	Which trips are valid? Five checks, e.g. a fare above zero; count the trips that fail each check.	Like every operation, it reads and checks the whole year.
M2 demand by hour	How many trips start in each borough, weekday and hour, and what are their average fare, duration and distance?	One pass over all trips, which each task first sums up locally.
M3 zone statistics	How does each zone change from month to month, and which months are unusual for it?	It first sums all trips into one row per zone and month (3,083 rows); its window functions on that small table are cheap (0.56 s laptop, 1.0 s cluster).
O1 unusual fares	Which trips cost far more per kilometre than similar trips (same pickup zone, same hour), i.e. above the group's mean + 3σ?	The mean and σ of each group are computed with a window, so the trips of a group must meet in one task: every trip is regrouped across tasks (a *shuffle*), sorted, and partly written to disk. The largest group has 169,582 trips.
O3 demand features	How busy is each zone in each hour, compared with its usual level at that weekday and hour, and with the same hour four weeks earlier?	The counts per zone and hour (2,303,880 rows) are joined with a copy shifted by four weeks (672 hours). Spark picks the join method from a size estimate [2]: 9.58 MB on a one-month run, just under the 10.49 MB limit for copying a table to every task; 122.44 MB on the year, where Spark therefore regroups and sorts both sides.

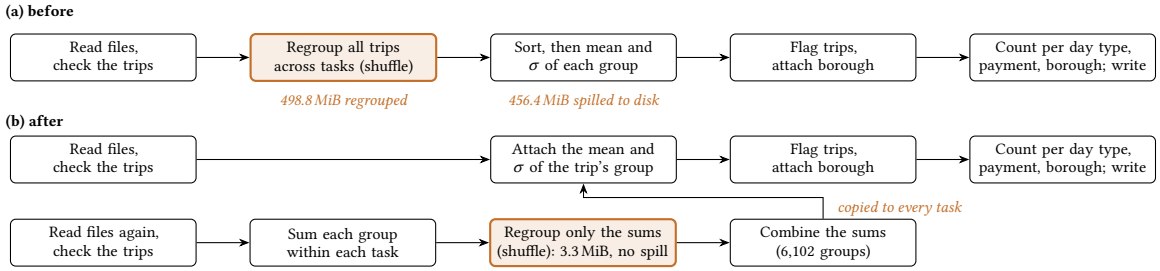


Figure 2: Final plan of the unusual-fare operation (O1), 2025, laptop, before (a) and after (b) the change; simplified from the plans Spark recorded. Read each row left to right; the outlined boxes are where data is regrouped across tasks. After the change only small sums are regrouped, and the trips are read twice: the lower row runs first, and its result is copied to every task of the upper row.

Table 2: The two test machines.

	Laptop	Simulated cluster
what it is	Spark directly on the laptop	Spark master and workers in Docker containers on the laptop, linked by a virtual network
hardware	Apple M5, 10 cores (4 performance, 6 efficiency), 24 GB	the same laptop, via a VM of 10 CPUs and 11.73 GiB
workers	one process, 10 cores	2 workers of 2 cores (2–5 in Section 5), 1024 MiB each
software	Spark 4.2.1 and Java 21 on both	

Mode was on), so the old and new versions always ran in alternation, in one session, from one build. A warm-up run was discarded, and medians are reported.

What did the measurements show? Table 3 gives the baseline: 25.94 s on the laptop and 34.77 s on the simulated cluster.

- The unusual-fare operation (O1) was the slowest (7.79 s on the laptop).
- It was also the only operation that spilled data to disk (456.4 MiB).
- The tasks spent 71 % (laptop) and 88 % (cluster) of their run time computing on a processor, and 3.5 % and 5.4 % in garbage collection, so reading from disk and memory cleanup were not the main costs.

To see which parts decide the total time, Figure 3 splits the baseline's critical path. The five reading stages (each also checks

Table 3: Baseline (before) and final pipeline (after), 2025, 7 alternating pairs, medians in s (ranges in Table 5).

	Laptop, 10 cores		Cluster, 2×2 cores	
	before	after	before	after
whole pipeline T	**25.94**	**21.70**	**34.77**	**31.74**
M1 / M2 / M3	4.52/3.00/5.14	4.37/2.86/4.93	5.86/4.50/7.21	5.86/4.33/7.56
O1 / O3	7.79/5.31	4.51/5.02	9.18/7.67	5.43/8.17
regrouped (MiB)	656.9	161.4	650.0	153.9
spilled to disk (MiB)	456.4	0	431.7–432.1	0

the trips and starts its operation) took about three fifths; O1's sort-and-average step about one sixth. These shares come from an earlier session whose times were about 1.6 to 1.7 times longer (41.19 s and 58.28 s), with the same plans, data volumes and task counts.

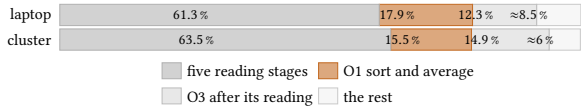


Figure 3: Where the baseline run time went (share of the critical path, earlier session).

Suspected causes. Table 4 lists the six suspected causes, each with the most it could save. Section 4 tests three; a fourth was dropped before measuring.

Table 4: Suspected causes, evidence, and the most each could save (share of the baseline run time, laptop / cluster).

	Suspected cause	Evidence, and what happened	Saves up to
1	The input is read again for each operation	829,973,299 B and 48,722,602 records read by each of the five operations; nothing kept in memory. Tested: change 1.	37.8 / 38.0 %
2	O1 regroups, sorts and averages all trips	498.8 MiB regrouped, 456.4 MiB spilled; tasks of even length (longest / median 1.03). Tested: change 2.	17.9 / 15.5 %
3	Idle cores in the last round of each reading	One task per file: 12 tasks on 10 cores (laptop), 10 on 4 (cluster). Not tested.	13.1 / 12.7 %
4	A text pattern extracted from the file path of every row (M1, M3)	M1 and M3 read more slowly than the others, but they also compute more. Not tested.	up to 11 %
5	O3 computes the same joined table twice	Two stages each read the same 32.9 MiB. Dropped: keeping it in memory grew O3 from 76 to 855 tasks (plan run).	5.2 / 6.4 %
6	O3's four-week join regroups and sorts both sides	64.2 MiB regrouped again on one side, 24.5 MiB on the other. Tested: change 3.	3.2 / 4.9 %

4 OPTIMISATION STUDY

Each change was tested alone, switched by the program option and measured as in Section 3. Every version printed the same results (O1's internal averages agree to within 10^{-12} relative). Table 5 lists every number.

4.1 Change 1 (cause 1): keep the cleaned trips in memory — undone

Bottleneck. Five operations read and check the whole year: about three fifths of the baseline. **Hypothesis.** If the cleaned trips are kept after the first use, the later operations need not read the files again. **Change.** After cleaning, the trips are stored in memory (and on disk when memory is full) and released after the last operation. **Data.** The later operations read nothing from the files, as intended (0 records instead of 48,722,602). **Time.** The pipeline took 32 % longer on the laptop (25.54 → 33.61 s) and 42 % longer on the cluster (34.21 → 48.64 s). **Reason.** Building the copy cost more than the reading it saved: M2, which fills the copy, became 10.11 s (laptop) and 15.24 s (cluster) slower, only M3 gained (3.61 and 4.68 s), and O3 became slower because it now read the copy twice. On the cluster the copy (4,828–5,689 MiB) did not fit in the workers' storage memory (434 MiB each), so it lived entirely on disk. **Verdict.** Undone. Keeping data in memory pays only if the copy fits and costs less to build than the reading it saves.

4.2 Change 2 (cause 2): sum first, then attach — kept

Bottleneck. O1 needs the mean and σ of the fare per kilometre of each group (one pickup zone in one hour). With a window, the trips of a group must meet in one task, so all 43.6 million trips were regrouped (498.8 MiB), sorted and partly spilled to disk (456.4 MiB): about one sixth of the run time. **Hypothesis.** If each task first sums its own trips per group, only small sums travel (at most 6,312 groups per task), and the sort disappears. **Change.** The sums are combined into one mean and σ for each of the 6,102 groups; that small table is copied to every task and attached to the trips (Figure 2b). **Data.** Only 3.3 MiB was regrouped (2.7 MiB on the cluster) and nothing spilled. The 4.84 s sort-and-average stage was replaced by a 0.03 s stage that combines the sums. **Cost.** The trips are now read twice, once for the sums; this extra reading cost 1.42 s (1.89 s on the cluster), less than the 4.1 s (6.5 s) that had been feared. **Time.** O1 fell from 7.77 to 4.80 s on the laptop (−38 %) and from 9.36 to 5.53 s on the cluster (−41 %); the ranges of the repeats did not overlap. In the final measurement (Table 3) O1 takes 42 % (laptop) and 41 % (cluster) less time, the whole pipeline 16 % and 9 % less, and the new version was faster in all 14 pairs. **Verdict.** Kept.

4.3 Change 3 (cause 6): copy the shifted table to every task — undone

Bottleneck. O3's join of the count table with its shifted copy regroups and sorts both sides, and regroups 64.2 MiB a second time. **Hypothesis.** Spark over-estimates the shifted copy (122 MB, for 2.3 million rows of 3 columns); if it is copied to every task instead, the join stage and the second regrouping disappear. **Change.** A hint tells Spark to copy the shifted table to every task; the join itself stays. **Data.** O3's regrouped data fell from 156.5 to 67.8 MiB (−57 %) and one stage disappeared. **Time.** O3 did not get faster: 7.92 → 8.63 s on the laptop and 13.83 → 14.23 s on the cluster, with overlapping ranges. (This session ran slower overall for both versions; Low Power Mode was found on afterwards.) **Reason.** Building and copying the table made the next stage wait 0.96 s (1.32 s on the cluster), and that stage now also does the join; together this about matched the removed join stage (1.11 s and 2.86 s). The cause could save at most 3.2 % / 4.9 % of the run time, and O3 alone varied by 12.8–19.4 % between repeats, so 5 pairs could not resolve a gain of that size. **Verdict.** Undone: no measured gain, and the copied table (268,435,424 B) grows with the years and already did not fit in the cluster workers' memory.

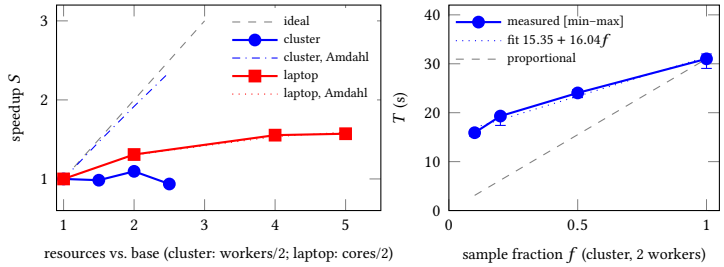
5 ANALYSIS

Two questions remain about the final program, which keeps only change 2. Each gets a short answer first, then the evidence (Figure 4).

Does the run time grow in proportion to the data? No. If it did, 10 % of the data would take 10 % of the time; it takes 51.3 %. **Evidence.** On the cluster with 2 workers, T was 15.91, 19.31, 24.04 and 31.01 s with 10, 20, 50 and 100 % of the data (Figure 4, right). A straight line through these points, $T = 15.35 + 16.04f$ seconds, puts about half of the full run time in a part that does not depend on the data size. Three causes explain this. (1) The sample is taken *after* the files are read, so every reading stage still reads all 48,722,602 trips; the first reading stage (M1) used 5.6 s of processor time at 10 % and 12.9 s at 100 %. (2) The number of tasks stays at 98–99, because it follows the files, not the sample, so the fixed cost of each task stays. (3) O3 works on a table of zones × hours, whose size does not depend on the trips; the regrouped data grew only from 117.3 to 187.2 MiB, mostly O3's. Memory is not a cause: no fraction spilled to disk, and garbage collection stayed at 6.2–6.9 %.

Table 5: Every change, tested alone. Median [min–max] of T in s. Compare only within a row: sessions differ in machine state. Run-ids per row are in `experiments/LOG.md`.

Change, verdict	Machine, pairs	before	after	ΔT	What else changed
1 Keep in memory, *undone*	laptop, 4	25.54 [24.90–27.05]	33.61 [31.92–38.91]	+31.6 %	records read later 48,722,602 → 0; M2 +10.11 s
	cluster, 3	34.21 [33.82–34.80]	48.64 [46.91–48.73]	+42.2 %	copy 4,828–5,689 MiB, all on disk
2 Sum first, ***kept***	laptop, 5	27.69 [25.86–28.09]	23.65 [22.77–25.24]	−14.6 %	regrouped 498.8 → 3.3 MiB; O1 −38.2 %
	cluster, 3	36.01 [33.57–36.49]	31.39 [31.23–31.75]	−12.8 %	regrouped 498.8 → 2.7 MiB; O1 −40.9 %
3 Copy table, *undone*	laptop, 5	34.97 [32.26–36.37]	35.23 [33.48–36.62]	+0.7 %	O3 regrouped 156.5 → 67.8 MiB; O3 +9.0 %
	cluster, 5	51.73 [51.23–54.37]	51.52 [50.24–53.48]	−0.4 %	O3 regrouped 149.6 → 64.9 MiB; copy fell back to disk
Final (change 2 only)	laptop, 7	25.94 [23.39–27.24]	21.70 [20.77–24.73]	−16.3 %	O1 −42.1 %; 7 of 7 pairs faster
	cluster, 7	34.77 [32.08–37.27]	31.74 [29.66–33.27]	−8.7 %	O1 −40.8 %; 7 of 7 pairs faster



slots	T_p (s)	S	E (%)	f	CPU (s)	CPU/T_p
cluster (2 slots per worker)						
4	29.52	1.000	100.0	–	71.8	2.43
6	29.99	0.984	65.6	1.047	90.4	3.01
8	26.95	1.096	54.8	0.825	108.6	4.03
10	31.55	0.936	37.4	1.114	128.6	4.08
laptop alone (slots = cores)						
2	35.93	1.000	100.0	–	64.8	1.80
4	27.45	1.309	65.4	0.528	81.3	2.96
8	23.12	1.554	38.9	0.525	127.0	5.49
10	22.85	1.572	31.4	0.545	131.6	5.76

Figure 4: Left: speedup of the simulated cluster (relative to 2 workers) and of the laptop alone (relative to 2 cores), with Amdahl's law [1] for comparison; the ideal line (S equal to the resources) is drawn up to 3. The cluster's Amdahl line uses its one-task stages (f = 0.043), the laptop's is a fit (f = 0.535). Middle: run time against the fraction of the data, 5 runs each. Right: the numbers behind the left plot; CPU = processor time of all tasks; runs per row: cluster 4, 3, 6, 3, laptop 3 each. All three panels are generated from `experiments/analysis/*.csv`.

Does the run time fall when more workers share the job? No, not on one laptop. The *speedup* S is the time with 2 workers divided by the time with p workers (ideal with 5 workers: 2.5); the *efficiency* E is S divided by its ideal. **Evidence** (Figure 4, right): with 3, 4 and 5 workers S was 0.98, 1.10 and 0.94; E fell to 37 %, and the cost (workers × seconds) grew from 59.0 to 157.8.

A first explanation: Amdahl's law. The law [1] says that the part of the work that cannot run in parallel (the serial fraction f) limits the speedup. Here that part is small: 12 one-task stages, 1.26 s, 4.3 % of the time, which would still allow $S = 2.35$ with 5 workers. If it were the cause, the f computed back from each measurement would stay near 0.043; it is near 1 and not steady (1.05, 0.83, 1.11). This points to an overhead that grows with the number of workers, not to a fixed serial part.

Why does the time not fall? The workers are containers on one laptop: a new worker adds task slots but no hardware. The same work costs 79 % more processor time with 5 workers (71.8 → 128.6 s), and the processors in use (CPU/T_p) level off at about 4, consistent with the laptop's 4 performance cores (which cores ran the tasks was not measured). The reading stages (about 70 % of the time) run their tasks in rounds; more slots cut the rounds from 3 to 2, but the median task became slower (1.14 → 3.00 s). Uneven tasks (the longest at most 2.16 times the median) and memory (no spill at any size) do not explain it.

And the laptop alone? The speedup is real but small: S = 1.31, 1.55 and 1.57 with 4, 8 and 10 cores relative to 2 (efficiency 65 %, 39 %, 31 %). Amdahl's model fits here ($f \approx 0.54$), but this fraction belongs to the hardware, not to the code: the one-task stages are only 3.0 % of the time, and the same work takes 2.0 times the processor time at 10 cores. With the same number of slots, the cluster is 7.5 % to 38.1 % slower than the laptop alone.

6 DISCUSSION

What remains. Reading the input is still the largest cost: the reading stages (now six, since O1 reads twice) take about 70 % of the run time on the cluster. Keeping only the needed columns in memory is untested; it might avoid the memory problem of change 1. Causes 3 and 4 were never measured, and O3 still computes the same table twice (cause 5).

Limits. (1) Every "node" is a container on one laptop, so the cluster study shows what happens when more workers share fixed hardware, not how a real cluster scales. (2) Single operations varied by 12.2–40.8 % between repeats, so effects of a few percent (such as cause 6) could not be resolved. (3) Sampling after the read makes the data-size result an upper bound on the time a smaller input would need. (4) Weak scaling (Gustafson [4]) was not testable, because the sample does not shrink the reading; and only 2025 is processed. **Next:** a real cluster, sampling by file so that the reading shrinks, and more years.

REFERENCES

[1] Gene M. Amdahl. 1967. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*. 483–485.

[2] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. 2015. Spark SQL: Relational Data Processing in Spark. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 1383–1394.

[3] ComPar 26/27. 2026. Lecture 7: Fundamentals of Parallel Performance; Lab 3: Spark Profiling. NOVA FCT course material.

[4] John L. Gustafson. 1988. Reevaluating Amdahl's Law. *Commun. ACM* 31, 5 (1988), 532–533.

[5] NYC Taxi and Limousine Commission. 2026. TLC Trip Record Data. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page. Yellow taxi 2025, 12 monthly Parquet files, downloaded 2026-11-04.

ACKNOWLEDGEMENTS

Claude Code (Anthropic's Claude models) assisted the work. It was used for coding and generating code, for discussing decisions, for guiding the work, and for gathering metrics and logs and helping to analyse them. It also assisted in drafting and editing the text of this report.

INDIVIDUAL CONTRIBUTIONS

Member	Work	Share
Radesh Govind	All tasks	100 %

COMMENTS AND SUGGESTIONS

None.